

PROVA PRÁTICA

IDENTIFICAÇÃO DO ALUNO

NOME			TURNO	Vespertino
UNIDADE	GAMA	TAGUATINGA	SOBRADINHO	☒ SIG
Itinerário:				
☒ DESENVOLVIMENTO DE SISTEMAS				
DESIGNER GRÁFICO				
JOGOS DIGITAIS				

- Esta prova prática tem como objetivo avaliar suas habilidades no Módulo 3 - Python com banco de dados.
- Leia atentamente cada questão antes de começar a executar as tarefas.
- Você dispõe de 240 minutos para completar todas as questões.

Técnico em Desenvolvimento de sistemas

Contexto:

A Clínica Veterinária VidaPet precisa de um sistema simples para registrar seus tutores de animais, os animais atendidos e os atendimentos realizados. Você foi contratado para desenvolver esse sistema usando Python com SQLAlchemy e SQLite.

O sistema não precisa ter interface gráfica nem menus interativos. O objetivo é criar a estrutura do banco, inserir dados reais e demonstrar que tudo funciona corretamente.

Parte 1 — Estrutura do Projeto (10 pontos)

Crie uma pasta chamada clinica_vetpet com a seguinte organização de arquivos:

```
clinica_vetpet/  
  app/  
    __init__.py  
    database.py  
    models.py  
    crud.py  
  venv/  
    main.py  
    requirements.txt  
    .gitignore
```

Cada arquivo tem uma responsabilidade específica, conforme estudado nas aulas:

database.py — configuração da conexão com o SQLite e criação da sessão.

models.py — definição das três entidades como classes Python (modelos ORM).

crud.py — funções de inserção para cada entidade.

main.py — script principal que cria as tabelas, insere dados e imprime confirmações.

Instale e ative a venv.

Instale o sqlalchemy.

Parte 2 — Entidades do Banco de Dados (25 pontos)

Implemente os três modelos abaixo em models.py. Os nomes dos campos, tipos e restrições devem ser respeitados exatamente como especificado:

Entidade	Campo	Tipo Python / SQL	Observação
Tutor	id	Integer / PK	gerado automaticamente
	nome_completo	String(100)	obrigatório
	telefone	String(20)	opcional
	email	String(100)	único
Animal	id	Integer / PK	gerado automaticamente
	nome_animal	String(60)	obrigatório
	especie	String(40)	ex: cão, gato, ave
	raca	String(60)	opcional
	peso_kg	Float	obrigatório
	tutor_id	Integer / FK	FK -> tutores.id
Atendimento	id	Integer / PK	gerado automaticamente
	data_atend	String(10)	formato DD/MM/AAAA
	motivo	String(200)	Obrigatório
	valor_cons	Float	valor da consulta
	animal_id	Integer / FK	FK -> animais.id

Observações importantes:

Não altere os nomes dos campos, eles serão verificados na correção.

Os campos marcados como "obrigatório" devem ter nullable=False.

As chaves estrangeiras devem usar o nome exato da tabela referenciada (verifique o `__tablename__` de cada model).

O campo email do Tutor deve ter unique=True.

Parte 3 — Funções de Inserção (crud.py) (25 pontos)

Implemente uma função de inserção para cada entidade:

```
inserir_tutor(db, nome_completo, telefone, email)
inserir_animal(db, nome_animal, especie, raca, peso_kg, tutor_id)
inserir_atendimento(db, data_atend, motivo, valor_cons, animal_id)
```

Cada função deve seguir o padrão: add -> commit -> refresh -> return.

Parte 4 — Script Principal (main.py) (15 pontos)

No main.py, após criar todas as tabelas com metadata.create_all(), insira os seguintes dados usando as funções do crud.py:

Tutores (minimo 2):

Tutor 1: use seu próprio nome, telefone e email fictícios.

Tutor 2: use o nome de um colega de turma (com permissão), telefone e email fictícios.

Animais (minimo 3, distribuídos entre os tutores):

Ao menos um de especie "cão", um de especie "gato" e um de especie livre.

Cada animal deve estar vinculado a um dos tutores inseridos.

Atendimentos (minimo 3, um por animal):

Cada atendimento deve ter motivo descritivo (ex: "Vacina anual V8") e valor realista.

Use a data de hoje no formato DD/MM/AAAA.

Ao final, imprima no terminal uma mensagem de confirmação para cada etapa:

```
print(f"Tutor inserido: {tutor.nome_completo} (id={tutor.id})")
print(f"Animal inserido: {animal.nome_animal} - {animal.especie}")
print(f"Atendimento: {atend.motivo} | R$ {atend.valor_cons:.2f}")
```

Parte 5 — Configuração do database.py (10 pontos)

Configure o arquivo database.py com SQLite seguindo o padrão estudado em aula:

URL no formato: sqlite:///./clinica_vetpet.db

autocommit=False e autoflush=False na sessão (SessionLocal).

check_same_thread=False nos connect_args do engine (necessário para SQLite).

Classe Base criada com DeclarativeBase.

Não é necessário instalar nenhum driver adicional, o SQLite já vem incluído no Python.

Parte 6 — Relatório Final (15 pontos)

Responda com suas próprias palavras. Respostas copiadas identicamente entre colegas serão anuladas. Faça o relatório usando o Microsoft Word.

- 1) O que é autocommit no SQLAlchemy e por que o valor padrão recomendado é False?
- 2) Qual a diferença entre db.add() e db.commit()? O que acontece se você chamar commit() sem ter chamado add() antes?
- 3) Por que o __tablename__ de um model precisa ser idêntico ao nome referenciado em ForeignKey()?
- 4) No projeto você usou String(60) para o campo raca. Por que não usar apenas String sem limite de tamanho?
- 5) O que é o arquivo .db gerado pelo SQLite após executar o projeto? O que acontece com ele se você deletar e rodar o main.py novamente?

Em caso de dúvida técnica durante a aula, consulte o professor, não consulte ferramentas de IA.

Forma de Entrega

Entrega via github:

Link do seu repositório chamado **clinica_vetpet** com todos os arquivos .py desenvolvidos.
Um print do terminal com a saída ao executar o main.py com sucesso.

Entrega via google classroom — relatório:

Responda as 5 questões da Parte 6, em um arquivo docx.
O relatório deve ser entregue junto com o link do repositório no mesmo dia.



Bom Trabalho!